

Transport Layer Security

Cloud Security

CSE4059

Certificates Are Not Enough

- Suppose the certificate chain is valid
- What is still missing?
 - Does the certificate identify the service the client requested?
 - Can the current server prove possession of the corresponding private key?
 - How can the endpoints protect the application data they exchange?

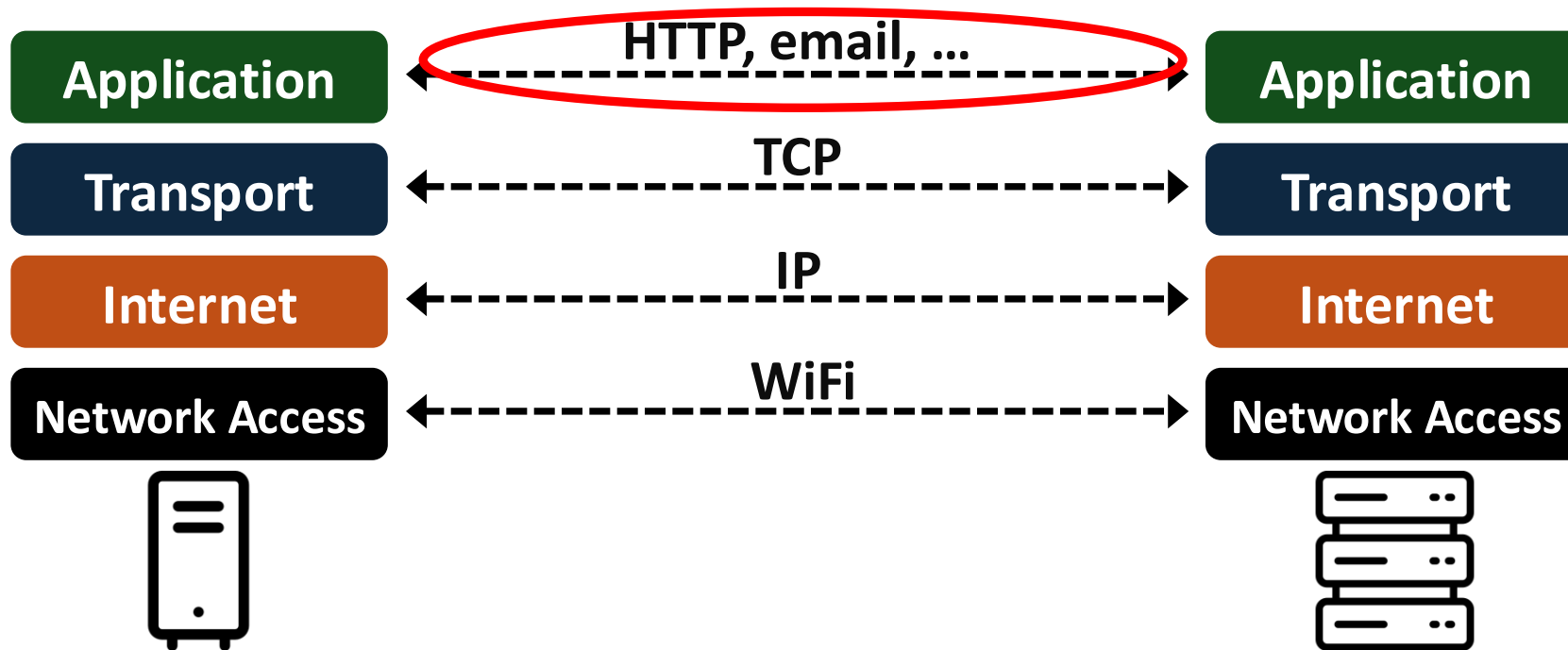
→ **Transport Layer Security (TLS)**

Transport Layer Security (TLS)

- TLS is a cryptographic protocol that secures communication between applications over a network
 - De facto standard for Internet security
- TLS typically authenticates the server and protects data confidentiality and *integrity*
 - Encrypts content and detects unauthorized modification
 - Commonly uses PKI and certificates for server authentication
 - Confidentiality, Integrity, and Authentication
- TLS protects data in transit between its endpoints, not plaintext inside a (compromised) endpoint

Application-Level Protection

- TLS protects application-level communication
 - Used to protect application traffic such as HTTP, email, etc.
 - Does not secure lower-layer protocols themselves (e.g., IP or TCP)



Evolution of SSL/TLS

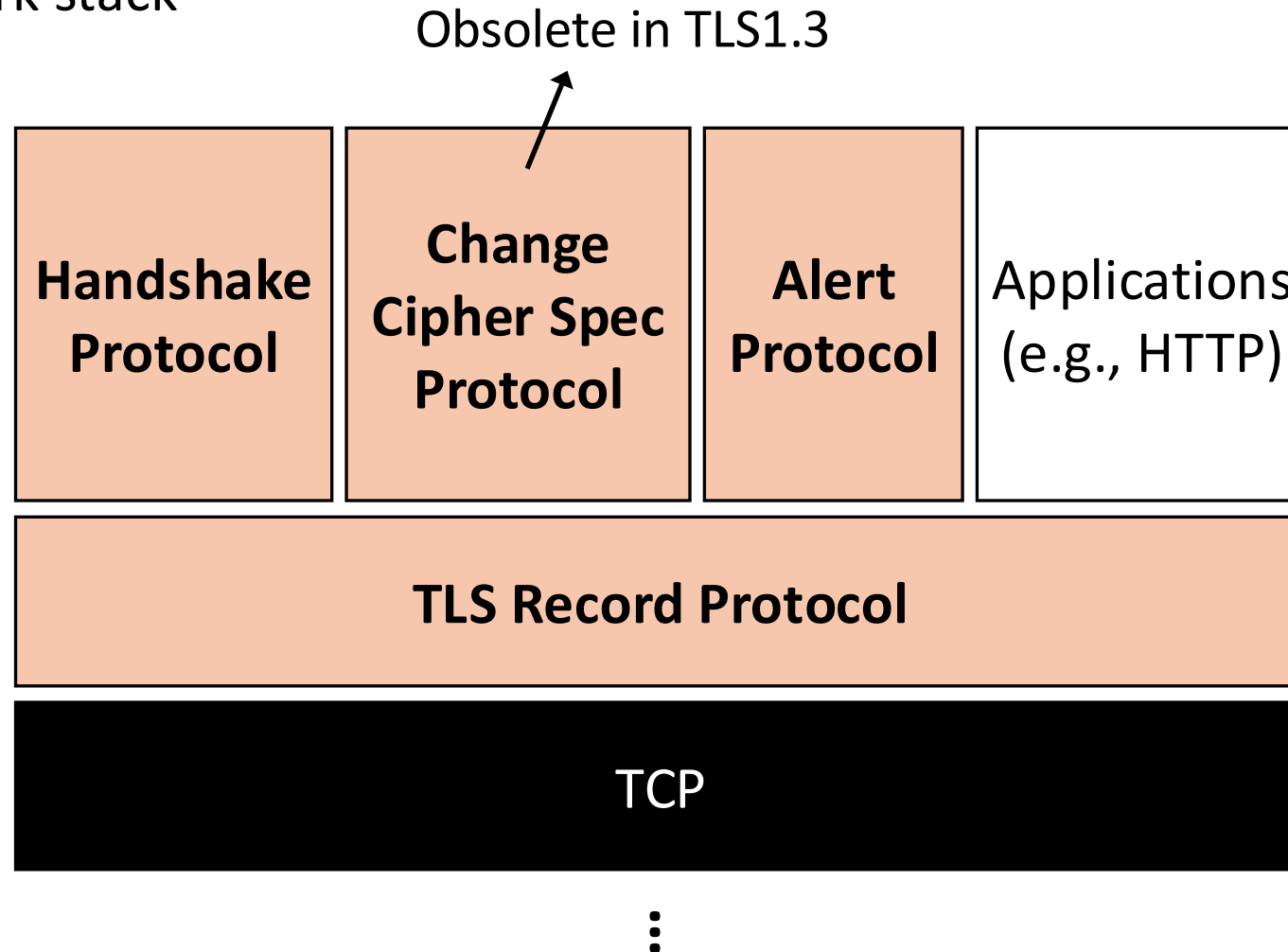
- TLS evolved from SSL and became the standardized successor
 - SSL (Secure Sockets Layer): the predecessor to TLS, originally developed by Netscape
- SSL 1.0: Internal Netscape design, never publicly released
- SSL 2.0: 1994, published by Netscape, several security weaknesses
- SSL 3.0: 1996, Designed by Netscape and Paul Kocher
- TLS 1.0: 1999, Internet standard based on SSL 3.0
- TLS 1.1: 2006
- **TLS 1.2**: 2008
- **TLS 1.3**: 2018, RFC 8446
- Today, TLS 1.2 and TLS 1.3 are the relevant versions

TLS Handshake and Record Protocols

- TLS has two core components
 - Handshake and Record protocol
- **Handshake protocol**
 - Negotiates protocol parameters and cryptographic algorithms
 - Authenticates the server
 - Establishes shared secret material and derives traffic keys
- **Record protocol**
 - Carries handshake messages, alerts, and application data in units called records
 - Once protection is active, uses the negotiated keys to provide confidentiality and integrity

TLS Protocol Architecture

- TLS in the network stack



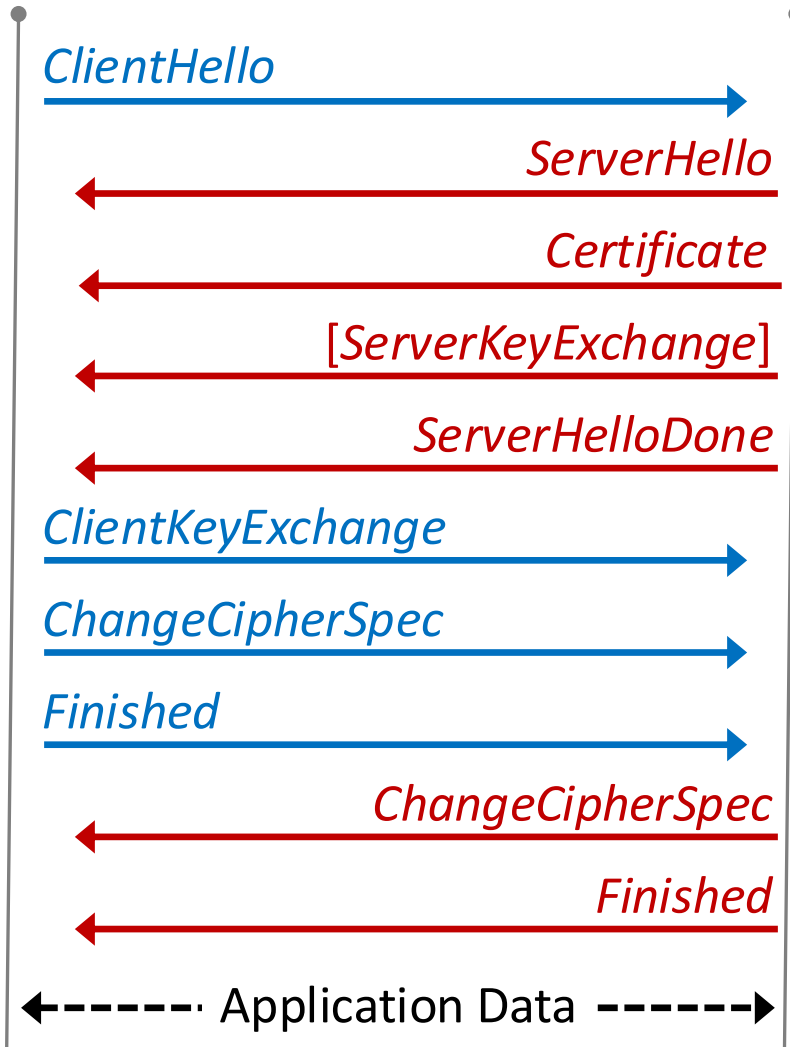
TLS Handshake Overview

- We focus on a full TLS 1.2 handshake with server authentication
- Negotiate a compatible TLS version and cryptographic algorithms
 - Interoperability between different implementations of the protocol
- Authenticate the server
 - The server presents its certificate; the client validates the certificate chain and verifies the server identity
 - Client authentication is optional
- Establish shared secret material and derive the keys used to protect records
 - The exact method depends on the selected key exchange

TLS 1.2 Handshake Overview

Client

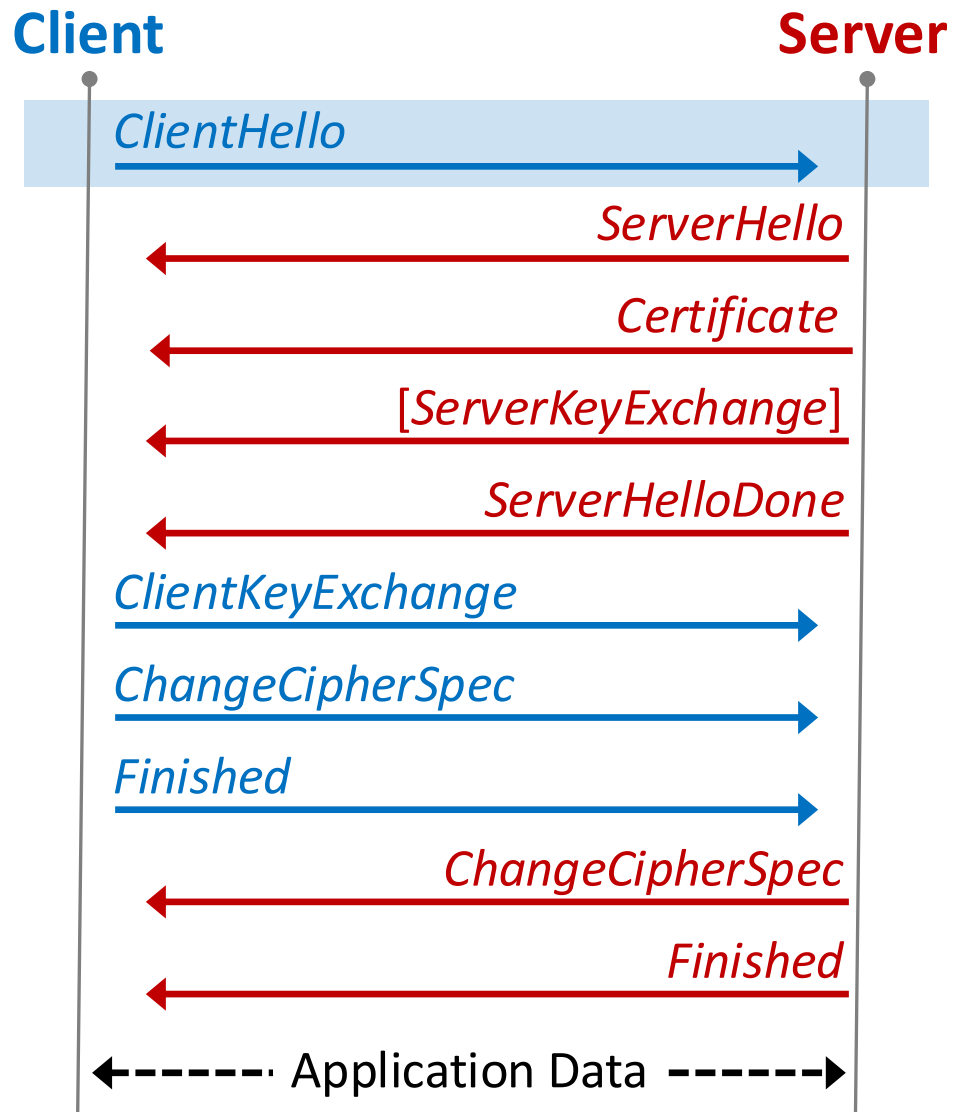
Server



- The *hello* messages negotiate the version and cryptographic algorithms
- The server sends its certificate and any required key-exchange parameters
- *ClientKeyExchange* supplies the client's key-establishment input
- *ChangeCipherSpec* (CCS) activates protection in one direction
- *Finished* checks the handshake using shared secret material
- After the handshake completes, application data is encrypted using the negotiated keys

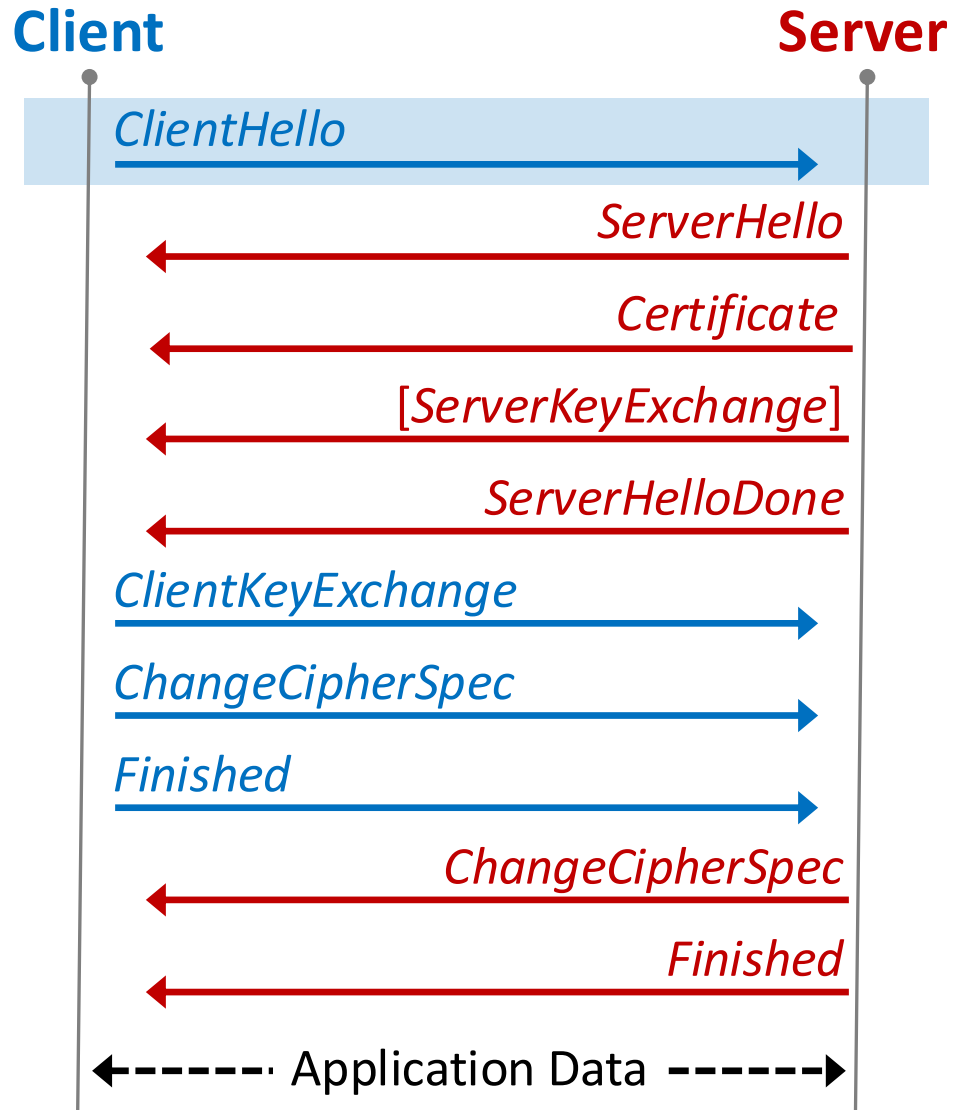
[Bracket]: method-dependent messages

TLS 1.2 Handshake – ClientHello



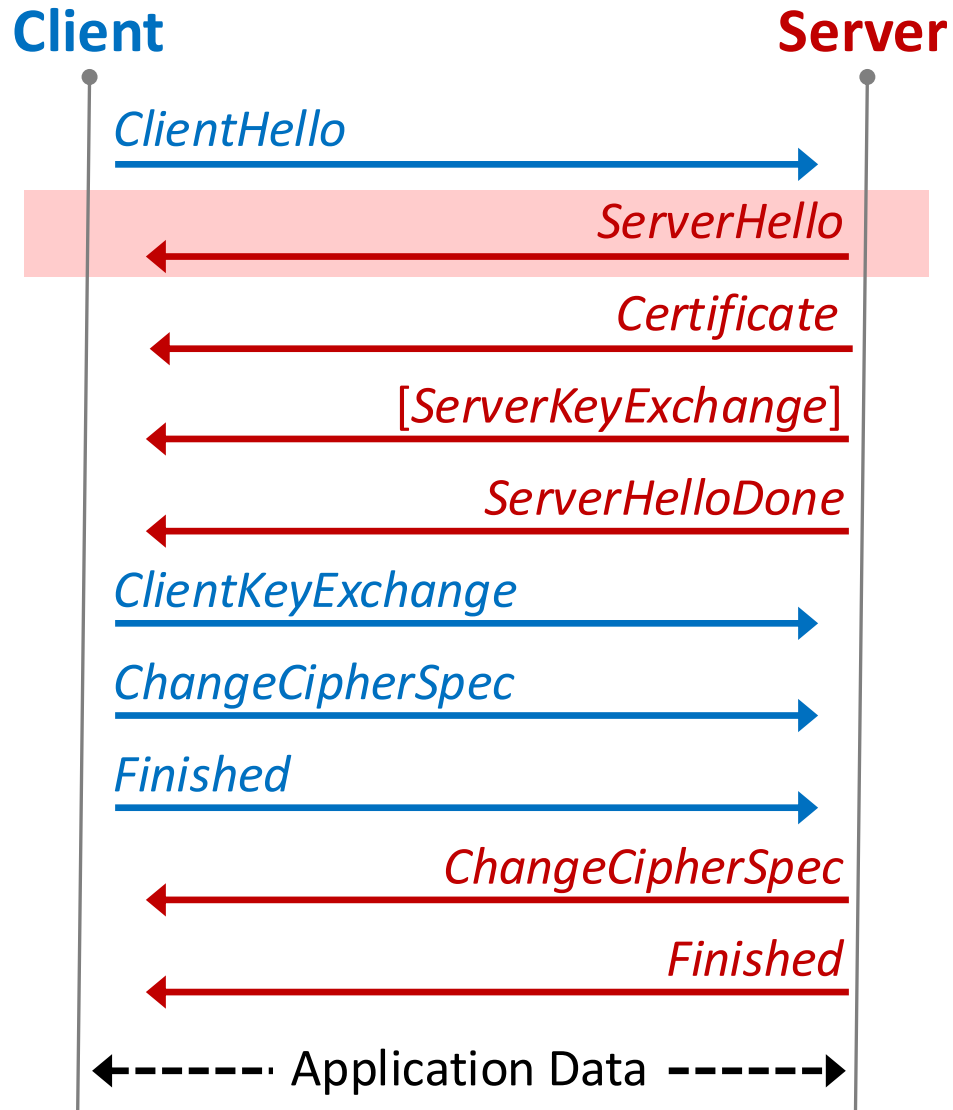
- *ClientHello* starts the handshake by offering a TLS version and supported cipher suites
 - A TLS 1.2 cipher suite identifies algorithms for key exchange, authentication, encryption, and integrity protection
- The client also sends a fresh client random value (ex. *client_random*)
 - It is public input to later authentication and key calculations
- *ClientHello* may also include extensions
 - ex) Server Name Indication (SNI)

TLS 1.2 Handshake – ClientHello



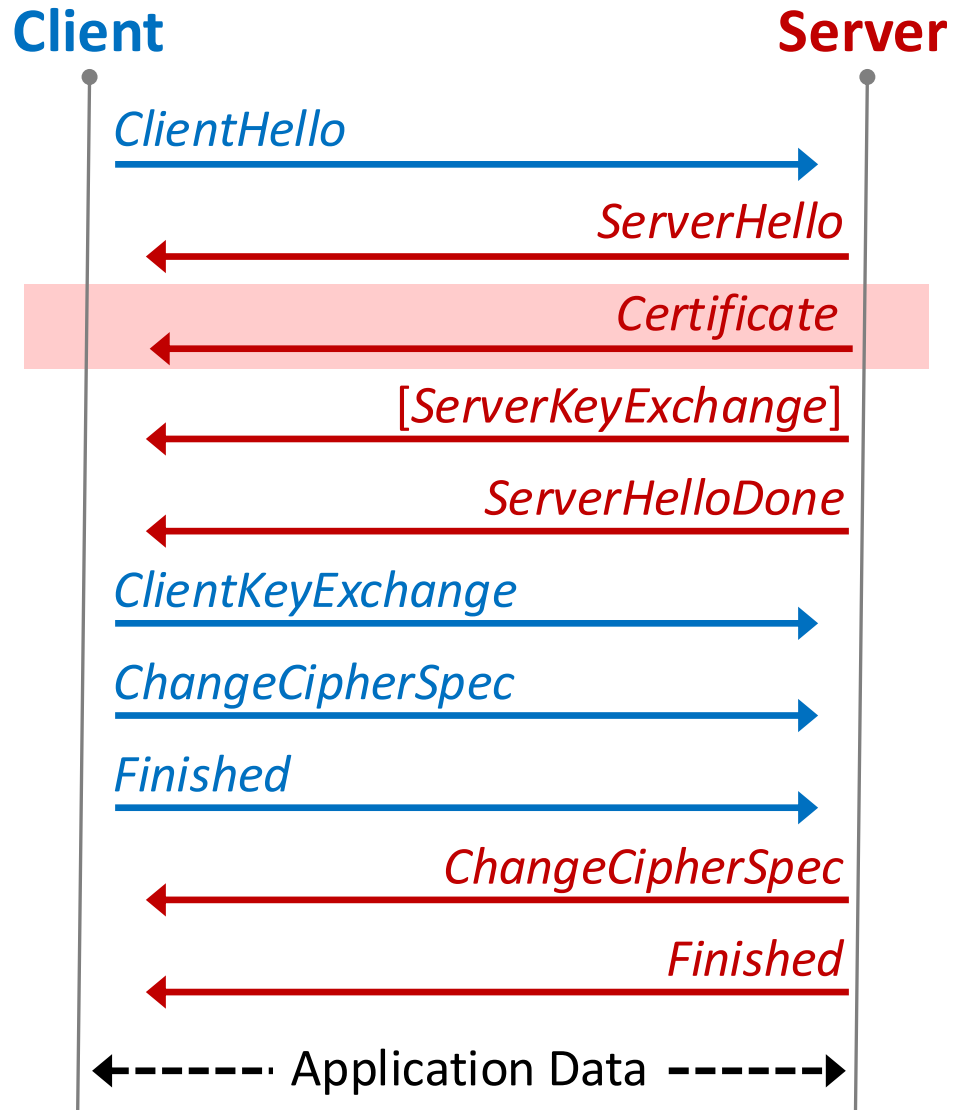
```
struct {
    ProtocolVersion client_version;
    Random random;
    SessionID session_id;
    CipherSuite cipher_suites;
    CompressionMethod compression_methods;
    Extension extensions;
} ClientHello
```

TLS 1.2 Handshake – ServerHello



- *ServerHello* returns the selected TLS version and cipher suite
 - The selection must be compatible with the client's offer
 - The selected cipher suite determines the later key-establishment method
- The server sends its own random value (ex. *server_random*)
 - The client and server random values are used in later key derivation and authentication calculations
- The *hello* messages do not authenticate the server

TLS 1.2 Handshake – Certificate

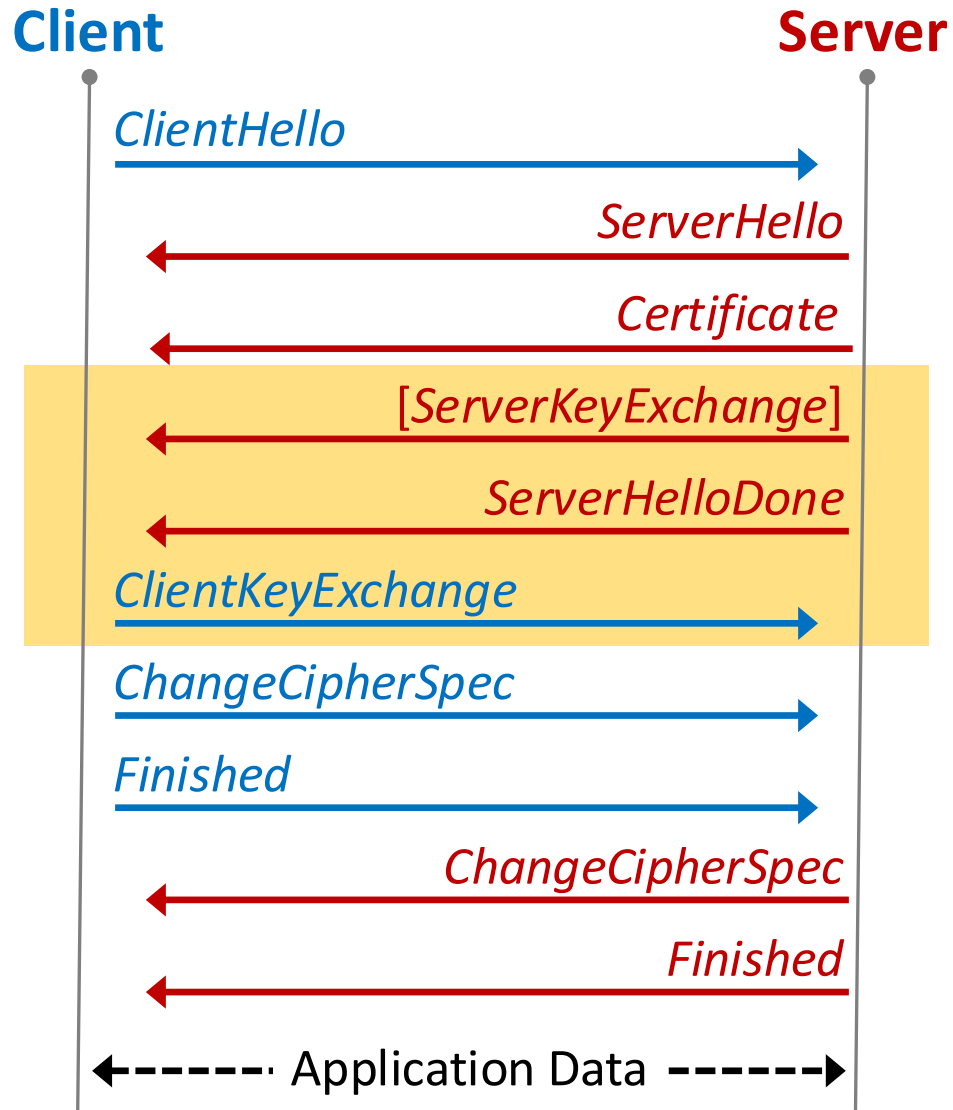


- The server sends a certificate chain
 - Its leaf certificate and needed intermediate certificates
- The client uses its own trust anchors and validates the certificate chain
 - e.g., Checks certificate signatures and validity periods
- A valid certificate chain alone does not complete server authentication
 - The client must also check whether the certificate identifies the requested service

Hostname Validation

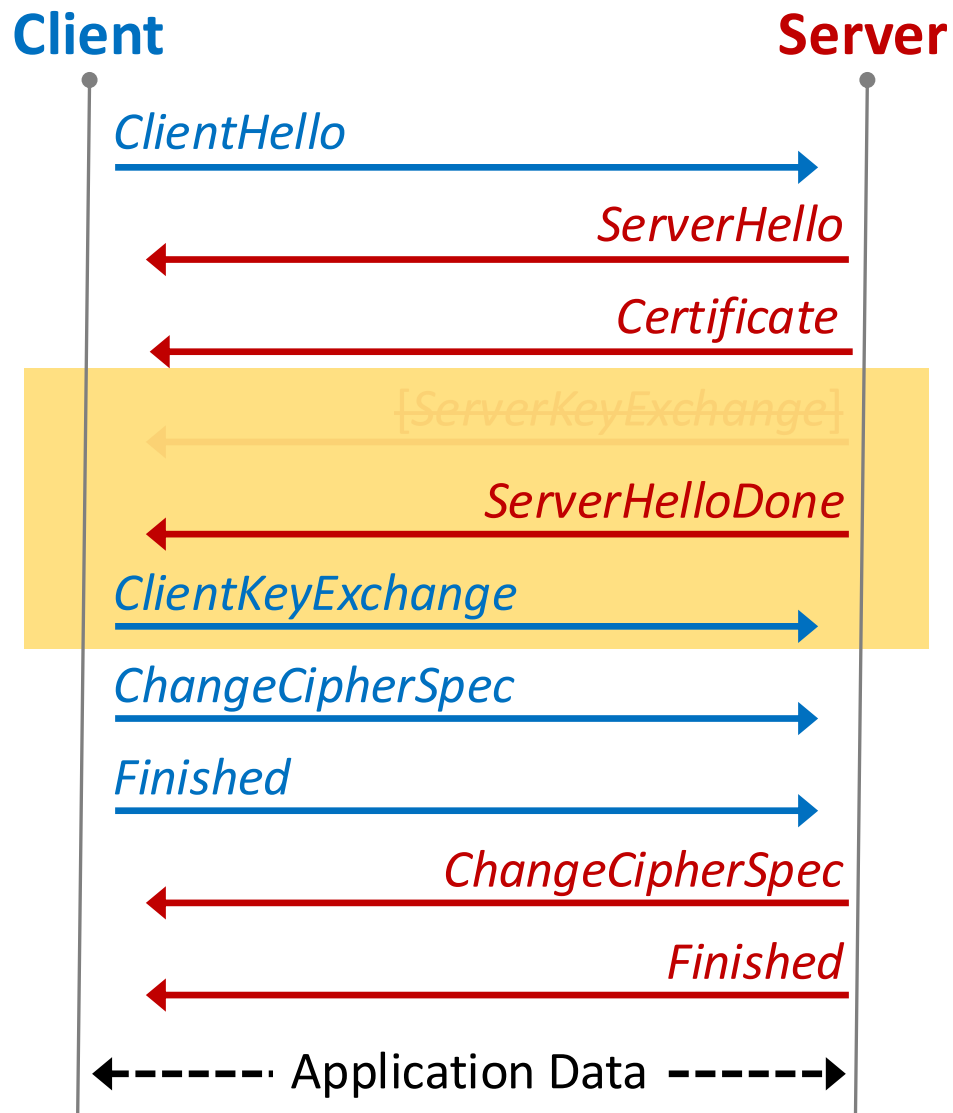
- **Hostname validation** checks whether the leaf certificate identifies the service the client requested
- The expected hostname comes from the application request
 - ex) *example.com* from *https://example.com*
 - The client matches it against DNS names in the leaf certificate (e.g., Subject Alternative Name)
- Example
 - Requested name: *example.com*
 - Leaf certificate' SAN: contains only *example2.com*
 - The client rejects the connection even if the certificate chain is valid
- A matching name alone does not prove that the current server possesses the corresponding private key
 - The remaining handshake provides this proof

Key Establishment in TLS 1.2



- TLS 1.2 supports different methods for establishing shared secret material
- RSA key transport
 - Client creates the premaster secret and encrypts it with the certificate's RSA public key
 - No *ServerKeyExchange*
- Ephemeral Diffie-Hellman (DHE)
 - Client and server jointly compute the premaster secret using ephemeral DH values

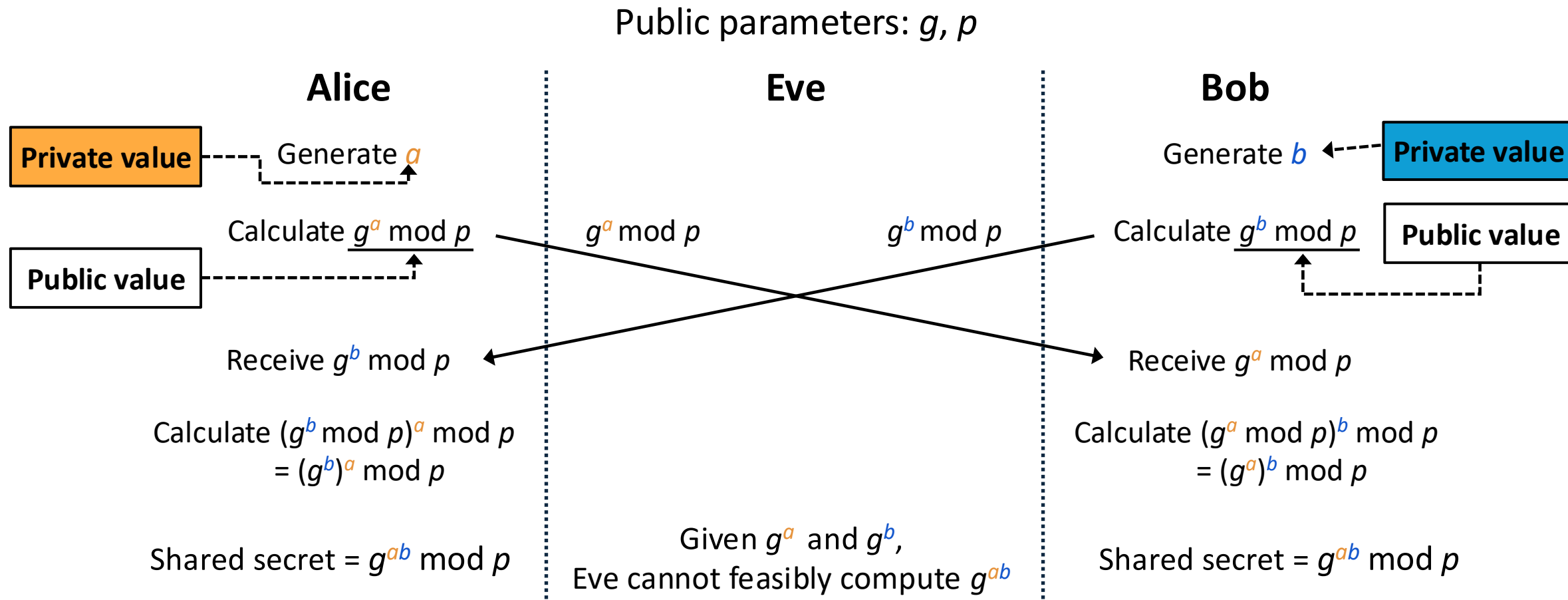
Key Establishment with RSA



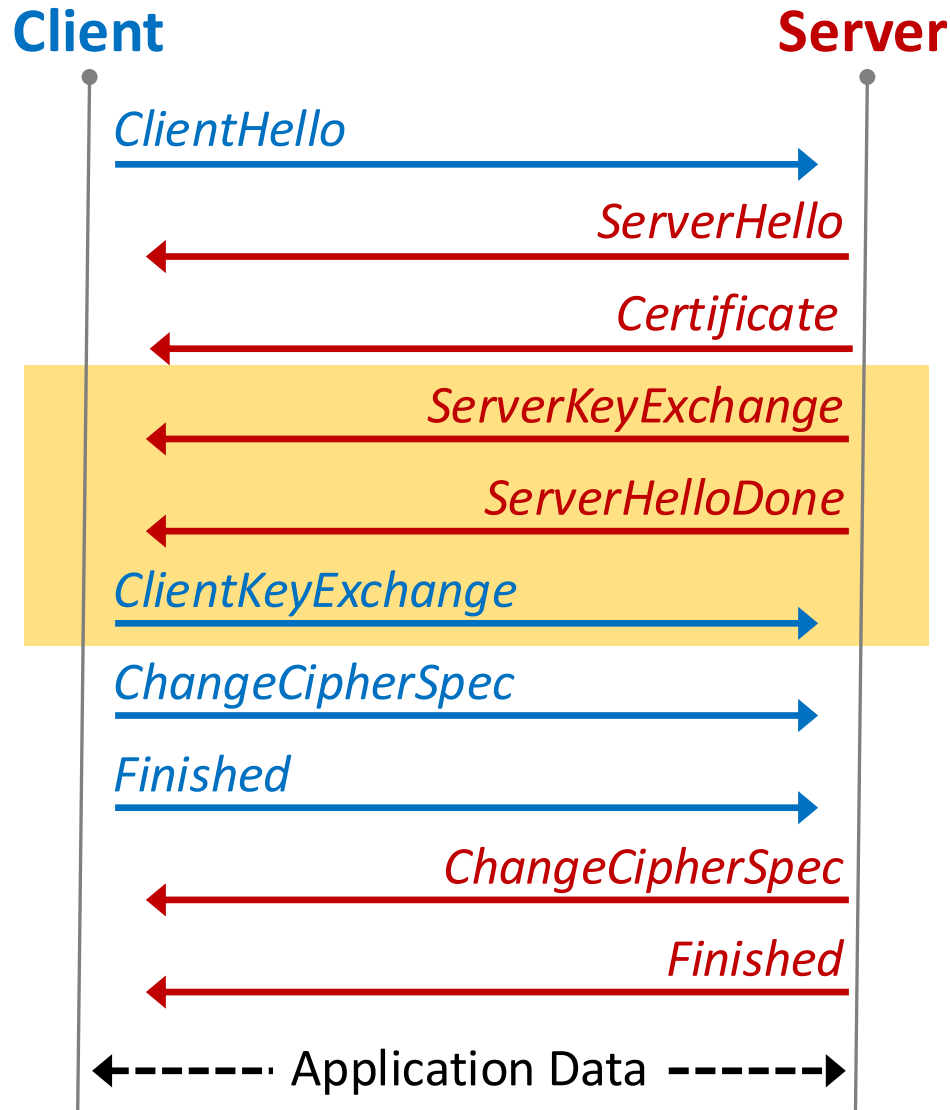
- *ServerHelloDone*: indicates that the server has finished sending its initial handshake messages
- Client
 - Generates the premaster secret
 - Shared secret material used as input to TLS key derivation
 - Encrypts this secret with the RSA public key in the server certificate
 - Sends the encrypted premaster secret in *ClientKeyExchange*
- Server
 - Decrypts it with the corresponding RSA private key
 - Both sides now have the same premaster secret

Diffie-Hellman Key Agreement

- Diffie-Hellman (DH) lets two endpoints compute a shared secret from private values and exchanged public values

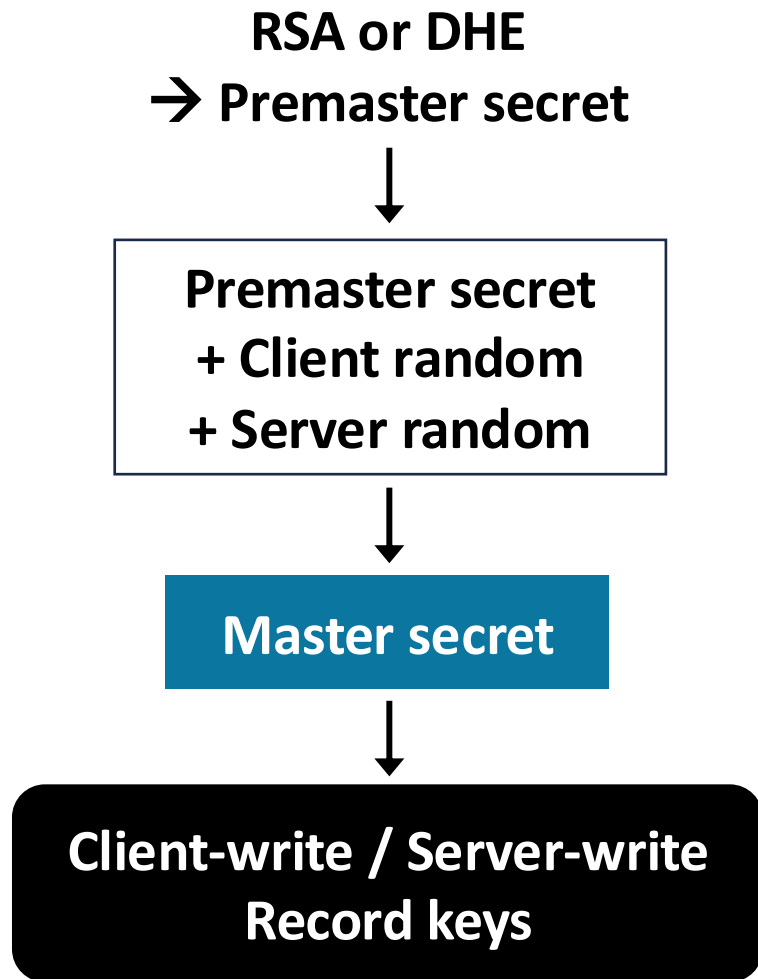


Key Establishment with Ephemeral DH (DHE)



- Ephemeral DH (DHE) uses fresh, temporary private values for the exchange
- Server
 - *ServerKeyExchange* carries the DH group parameters and the server's ephemeral public value (e.g., $g^b \bmod p$)
 - The server **signs** the hello randoms (*server_random*, *client_random*) and DH parameters
- Client
 - The client verifies the signature using the public key in the server certificate
 - After *ServerHelloDone*, the client sends its public value (e.g., $g^a \bmod p$) in *ClientKeyExchange*
- Both sides compute the same DH shared secret (e.g., $g^{ab} \bmod p$) → premaster secret

From Premaster Secret to Record Keys

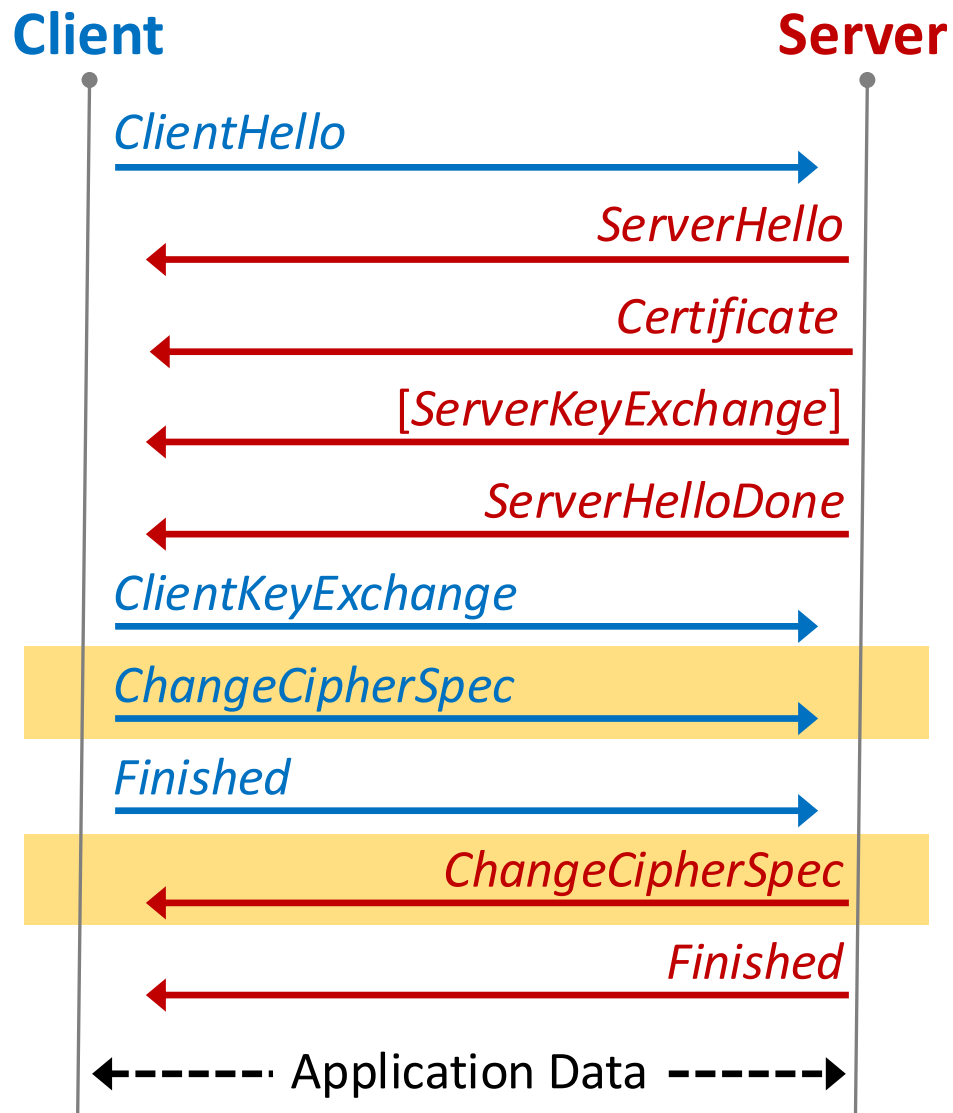


- RSA key transport and DHE both provide a premaster secret
 - They obtain it in different ways
- TLS derives the master secret from the premaster secret, client random, and server random
- The master secret and hello random values are used to derive separate record keys for each direction
 - These keys are derived locally and are not transmitted
 - The client-write key is used by the client to send and by the server to receive; the server-write key is used in the opposite direction
- These keys protect later application data

Forward Secrecy

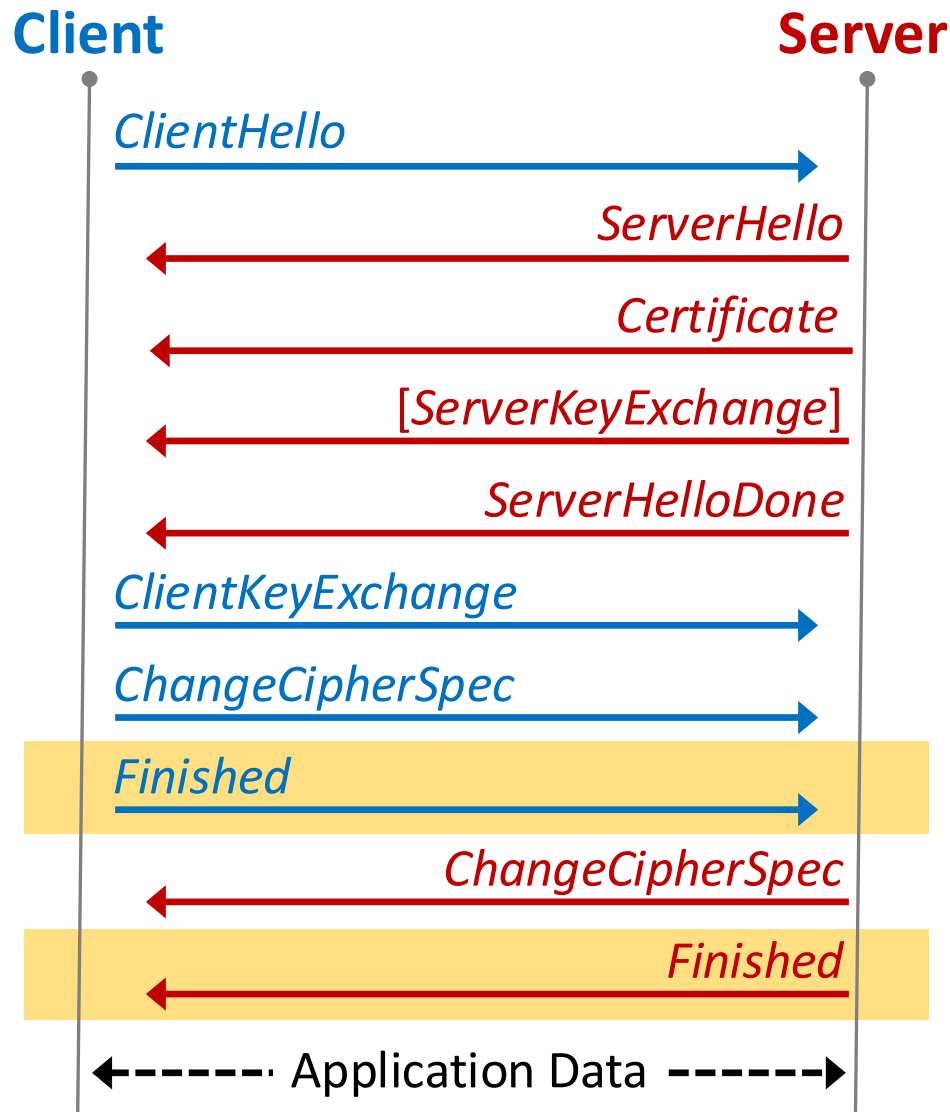
- **Forward secrecy:** compromise of a (long-term) private key does not compromise previously established session keys
- With RSA key transport
 - Recorded traffic includes the encrypted **premaster secret**
 - A later compromise of the RSA private key can reveal the premaster secret and the past record keys
 - No forward secrecy
- With ephemeral DH
 - A later compromise of the signing key does not reveal the earlier DH secret
 - Provides forward secrecy

ChangeCipherSpec



- By this point, both sides have derived the record keys
- *ChangeCipherSpec* (CCS) tells the peer that subsequent records from the sender will use the newly negotiated keys
 - Client CCS activates protection for the client → server direction
 - Server CCS activates protection for the server → client direction
- CCS itself is sent using the current (old) record state
 - Not protected by the newly derived keys
- CCS is a separate TLS protocol message carried by the Record Protocol, not a Handshake message

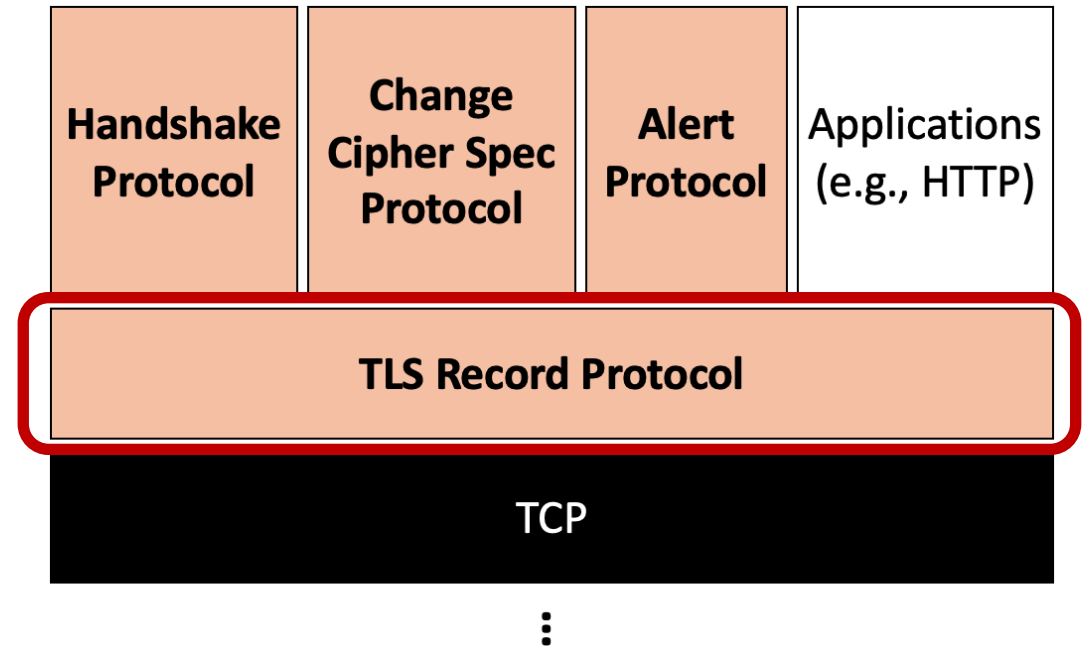
TLS 1.2 Handshake – Finished



- *Finished* verifies the integrity of the handshake and confirms that both sides derived the same master secret
 - Protected using the newly activated record keys
- The sender computes *Finished*
 - Using master secret, handshake transcript digest, ...
 - The handshake transcript is the ordered history of Handshake messages exchanged so far
 - It is hashed to produce a fixed-size digest
- The receiver independently computes the expected value and compares it with the received *Finished* value
 - If they differ, the handshake fails

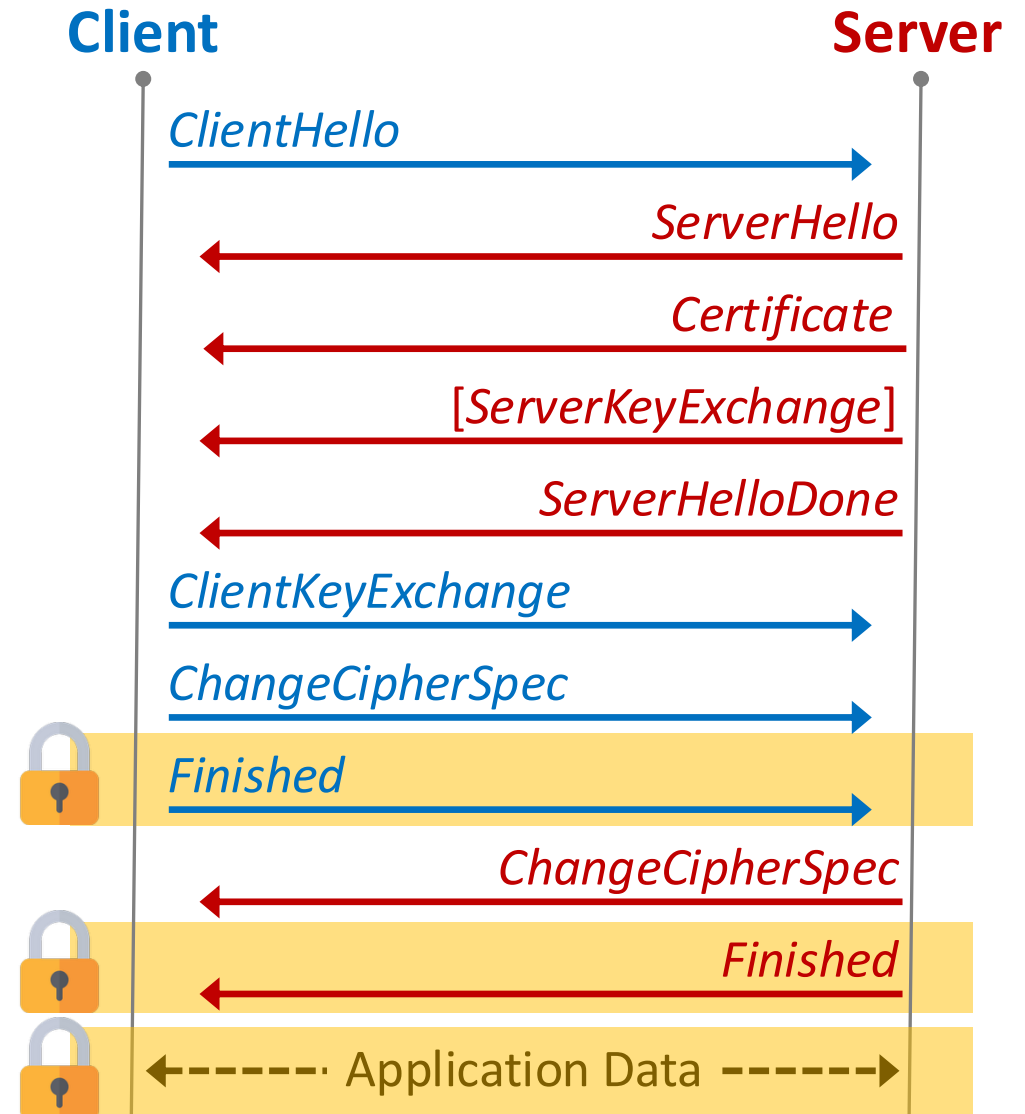
TLS Record Protection

- The Record Protocol carries TLS content in units called records
 - e.g., Handshake messages, application data
- After protection is activated, each record is protected using the negotiated record keys
 - Confidentiality: encrypts the record content
 - Integrity: detects unauthorized modification
- TLS uses separate keys for each traffic direction
 - Client → Server: client-write key
 - Server → Client: server-write key
- The receiver verifies the record protection before accepting its content



Problems in TLS 1.2

- Multiple round trips
 - Higher setup latency
- Legacy and weak cryptography
 - TLS 1.2 supports older options such as RSA key exchange
 - Some options do not provide forward secrecy
- Handshake privacy issue
 - Most initial handshake messages are sent in plaintext
 - A network observer can see handshake metadata
 - Server name
 - Supported and selected cipher suites
 - TLS versions and extensions

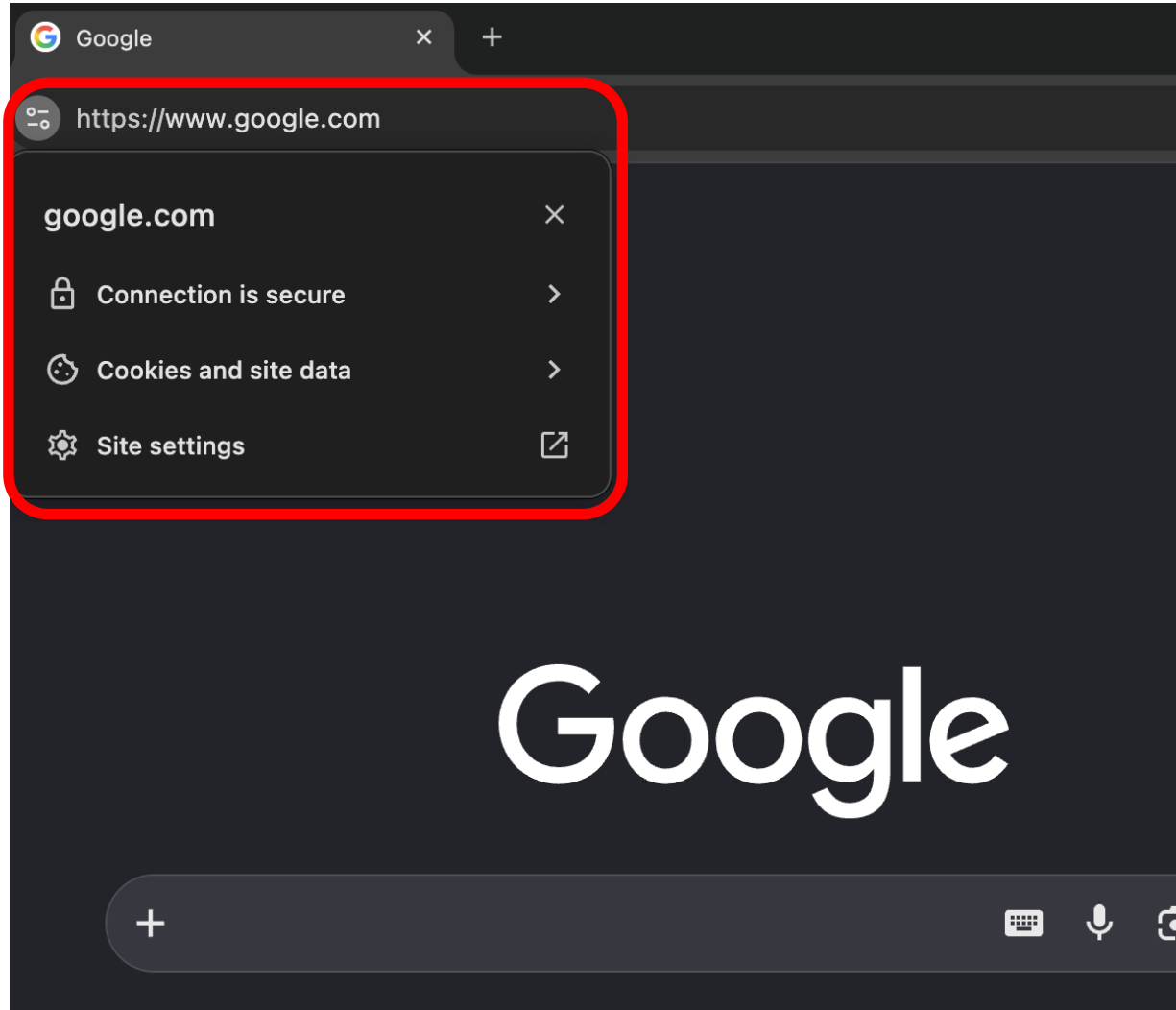


TLS 1.3

- Reduces setup delay
 - 1-RTT (for a full handshake), 0-RTT (possible for resumed sessions)
- Removes legacy key establishment options
 - No RSA key transport
- Encrypts more of the handshake
 - After *ServerHello*, subsequent handshake messages (ex. including the server certificate) are encrypted
 - *ClientHello* and *ServerHello* remains visible

TLS in the Real World

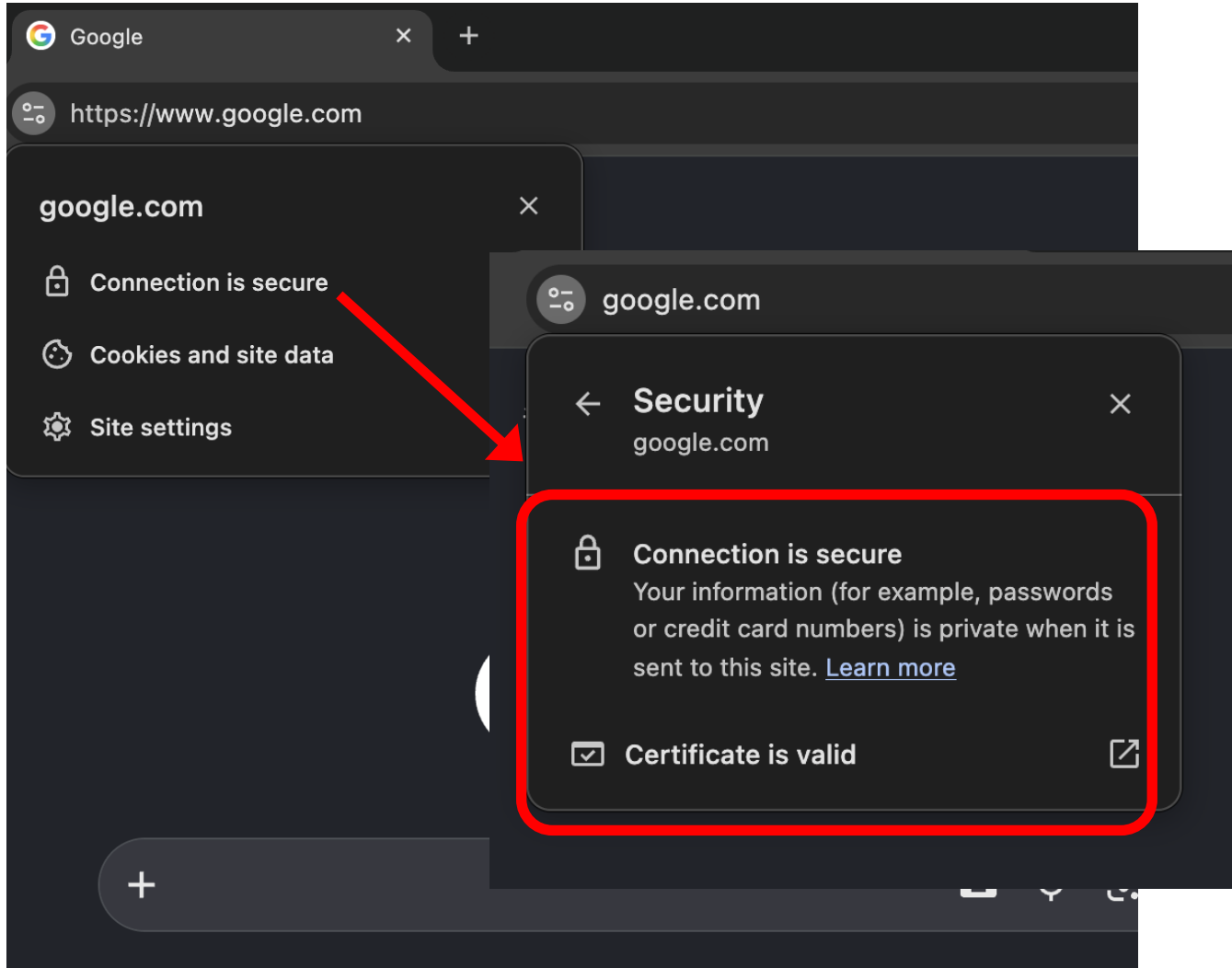
- HTTPS (HTTP over TLS) protects HTTP communication using TLS



<https://www.google.com/>

TLS in the Real World

- HTTPS (HTTP over TLS) protects HTTP communication using TLS



TLS in the Real World

- HTTPS (HTTP over TLS) protects HTTP communication using TLS

The image shows a web browser interface with a dark theme. The address bar displays 'https://www.google.com'. A sidebar on the left shows 'google.com' with options for 'Connection is secure', 'Cookies and site data', and 'Site settings'. A 'Security' overlay for 'google.com' is open, showing 'Connection is secure' and 'Certificate is valid'. A red box highlights the 'Certificate is valid' section, with a red arrow pointing to the 'Certificate Viewer' on the right.

Certificate Viewer: www.google.com

General Details

Issued To

Common Name (CN)	www.google.com
Organization (O)	<Not Part Of Certificate>
Organizational Unit (OU)	<Not Part Of Certificate>

Issued By

Common Name (CN)	WE2
Organization (O)	Google Trust Services
Organizational Unit (OU)	<Not Part Of Certificate>

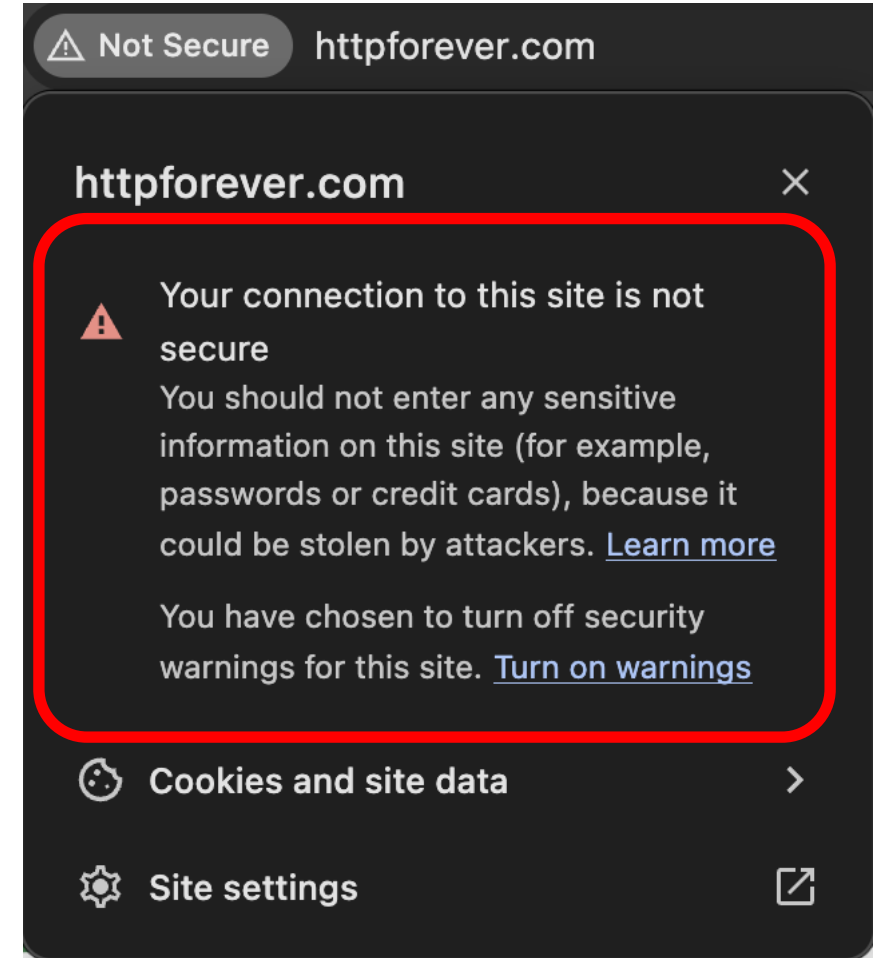
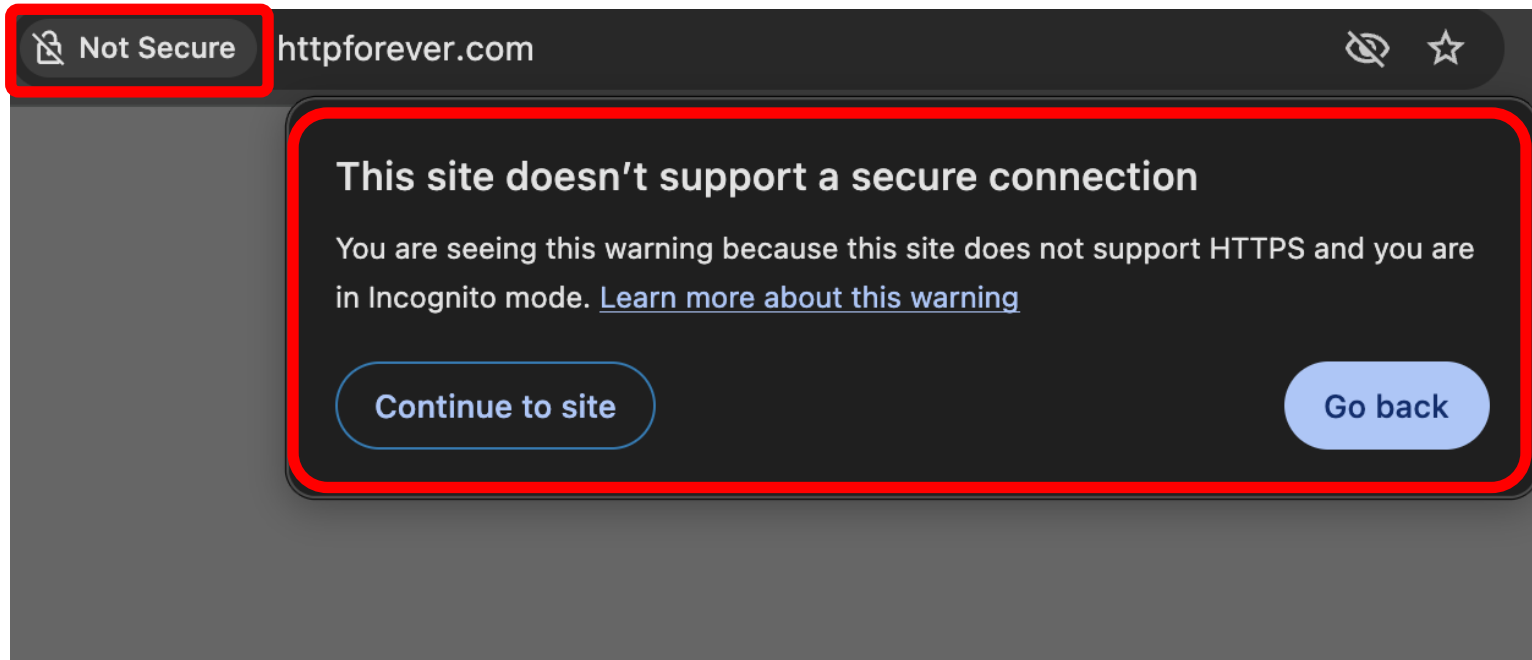
Validity Period

Issued On	Friday, September 4, 2026 at 5:06:59 PM
Expires On	Friday, November 27, 2026 at 5:06:58 PM

SHA-256

TLS in the Real World

- HTTP **without** TLS protection
 - Browsers may warn that the connection is not secure
 - HTTP traffic can be observed/modified by an attacker



Summary

- TLS secures communication between applications over a network
 - Provides server authentication, confidentiality, and integrity
- The TLS handshake sets up the secure connection
 - Negotiates security parameters
 - Authenticates the server
 - Establishes shared secret material and derives record keys
- The TLS record protocol carries and protects TLS data
 - Uses the derived keys to provide confidentiality and integrity
- TLS 1.3 simplifies and improves the TLS 1.2 design
 - e.g., faster handshake, more encrypted handshake messages